Kinematic Vector Symbolic Architecture:

Zero-Shot Causal Inference via Projective Geometric Algebra

Daniel Owen van Dommelen
Independent Research - WORKING DRAFT
theapemachine@gmail.com

April 29, 2026

Abstract

We present a four-layer computational architecture in which data, programs, and execution state occupy a single fixed-width binary frame, and learning emerges from the interaction of adaptive probabilistic tries, content-addressable routing, and collective eigenmode selection.

The fundamental primitive is a 1024-byte frame—128 sixty-four-bit words—partitioned into regions for token data, an embedded program, Boolean and geometric execution lanes, a 257-bit locality-sensitive affinity fingerprint, and structural link words. The low opcode nibble retains the complete set of two-input Boolean truth tables; the high nibble dispatches Projective Geometric Algebra operations over 64-byte multivector lanes held in the same frame. Frames are routed through a Kademlia-style distributed hash table where each node maintains a MarkovTrie: a suffix trie that learns online via surprise-modulated plasticity, adapting its own decay rate, learning rate, and other runtime parameters from signals it already computes.

A gossip protocol propagates compact digests of each trie's adaptive state across the network. From these digests, each node's Field detects emergent eigenmodes—clusters of structurally aligned, phase-coherent tries—and projects asymmetric pressure onto the local trie: amplifying learning in the dominant mode while suppressing it elsewhere. This mechanism functions as attention by differential survival, without matrices, softmax, or learned parameters.

The system processes all input modalities as raw byte sequences, requiring no learned encoders or modality-specific preprocessing. Continuous geometric state is used locally for transition motors, Procrustes alignment, and episodic re-alignment, while the affinity path remains a dense Boolean filter. A multi-substrate backend dispatches frame operations to CPU, CUDA, or Metal hardware, including native geometric kernels. We evaluate the architecture across text classification, code generation, image reconstruction, logical reasoning, and text generation. All experiments execute within the substrate and produce their own documentation artifacts, yielding the present paper as a structured output of the system itself.

1 Introduction

Most computational systems enforce a strict separation between three concerns: memory stores data, programs specify how data is transformed, and learning mechanisms adjust parameters to improve the transformations over time. This division is foundational in both classical software engineering and modern machine learning, where data resides in tensors, programs are defined by network architectures, and learning is driven by gradient descent over differentiable loss surfaces.

This work explores an alternative formulation in which these distinctions are collapsed. We describe a four-layer architecture in which data, programs, execution state, and learning coexist within a unified substrate:

1. **Values** (Layer 1): fixed-width 1024-byte binary frames carrying token data, a locality-sensitive affinity fingerprint, an embedded program, Boolean and geometric execution lanes, and structural links—simultaneously data, code, and execution context.
2. **MarkovTrie** (Layer 2): an adaptive probabilistic suffix trie that learns from every observation via surprise-modulated plasticity, requiring no training epochs, loss functions, or weight matrices.
3. **Kadabra DHT** (Layer 3): a Kademlia-style distributed hash table that routes Values to MarkovTries by affinity similarity, creating emergent specialization through content-addressable clustering.
4. **The Field** (Layer 4): a collective attention mechanism that detects emergent eigenmodes from gossip-propagated adaptive-state digests and projects asymmetric pressure onto each trie—amplifying learning in the dominant mode and suppressing it elsewhere.

Computation at Layer 1 is a two-lane in-frame ALU. The low opcode nibble applies the sixteen two-input Boolean truth tables across designated regions of one or more frames. The high opcode nibble dispatches Projective Geometric Algebra (PGA) operations—composition, sandwich application, and reversal—over 64-byte multivectors stored in the Context, Gradient, and Signals regions. Because programs and operands are stored within frames, there is no boundary between the instruction stream and the data it operates on; a frame may serve as passive storage, an executable program, a geometric motor, or all three.

Learning at Layer 2 is driven entirely by surprisal. The trie's learning rate, decay rate, prune threshold, classification context, temperature, beam width, interpolation weights, episodic blend, and unsupervised threshold are all derived online from signals the trie already computes—no hyperparameter search is required.

Routing at Layer 3 exploits a SimHash projection of the token content into the affinity region: five independent locality-sensitive hash projections using prime strides populate a 257-bit address so that Values with similar content are routed to the same network node, where a single MarkovTrie accumulates expertise in that content domain.

Attention at Layer 4 emerges from the interaction of gossip-propagated digests. Each node broadcasts a compact summary of its surprisal, growth rate, finite-field phase, and affinity vector. The Field detects clusters of nodes whose affinity vectors clear an adaptive Jaccard coupling threshold, then uses surprisal-velocity phase coherence and the global finite-field phase to modulate each node's decay and learning multipliers toward the dominant pattern.

The architecture does not employ gradient-based optimization or learned parameters. Its fast path remains dense Boolean logic—XOR distance, population count, and truth-table evaluation—but the current runtime also uses localized floating-point geometry for PGA motors, Orthogonal Procrustes alignment, and episodic drift correction. A multi-substrate backend routes both Boolean and geometric frame operations to whichever compute hardware is available—CPU, NVIDIA CUDA, or Apple Metal— through a unified kernel interface.

We evaluate the system across five experimental domains: text classification, multi-language code generation, CIFAR-10 image reconstruction, logical reasoning on the bAbI benchmark, and several text generation tasks including compositional assembly, out-of-corpus retrieval, and prose chaining. The experiments are defined programmatically, executed within the substrate, and compiled into the present document automatically.

The contributions of this paper are as follows. First, we define the four-layer architecture: the Value frame, the MarkovTrie, the Kadabra DHT, and the Field—and show how they compose into a functioning runtime that learns without gradients. Second, we provide the mathematical foundations: SimHash collision bounds, affinity coupling for eigenmode detection, surprise-modulated plasticity, PGA transition operators, and Orthogonal Procrustes alignment for cross-domain projection. Third, we describe the native kernel path that preserves the Boolean filter while adding CPU, CUDA, and Metal execution for the geometric lane. Fourth, we report empirical results across modalities, providing a baseline characterization of what this class of

architecture can and cannot achieve. Fifth, we demonstrate a self-documenting experimental pipeline in which the system produces both results and the corresponding research artifact.

This work does not aim to match the performance of large parameterized models on standard benchmarks. Its purpose is to explore a different point in the design space: one where retrieval and filtering reduce to deterministic, hardware-native bitwise operations; where continuous geometry is introduced only where it supplies an explicit spatial operator; where learning is local and surprisal-driven; and where attention emerges from collective dynamics rather than from learned projection matrices.

1.1 Representation and Primitive Operations

The system operates on a single primitive type called a *Value*: a fixed-width binary frame of 1024 bytes, organized as 128 sixty-four-bit words. Computation is expressed as transformations over Values using a hybrid ALU: dense Boolean truth tables for filtering and symbolic binding, and Projective Geometric Algebra operations for continuous transition and alignment.

Frame Layout. Each Value is partitioned into contiguous regions with distinct roles, specified by a configuration file that maps region names to word offsets and bit widths. The default layout is:

Table 1: Value frame layout. All offsets are word indices into the 128-word frame. Bit widths are given for the default configuration.

Region	Words	Bits	Purpose
Tokens	0–15	1024	Raw input bytes, Morton-packed token data
Program	16–23	512	32 packed instruction slots
Signals	24–31	512	Boolean ALU output or PGA result
Context	32–39	512	Context words or one 8-lane multivector
Gradient	40–47	512	Residual words or one 8-lane multivector
Meta	48–55	512	Adaptive metadata and runtime signals
Reserved	56–117	3968	Application-defined metadata
Kernel	118–119	128	Correlation and substrate residency words
Prev ID	120	64	Link to predecessor Value
Next ID	121	64	Link to successor Value
ID	122	64	Unique frame identifier
Affinity	123–127	257	5-word locality-sensitive fingerprint

The Token region holds the raw byte content of an input datum. The Program region contains executable instructions that the multi-substrate backend applies to the frame's own data. The Signals region receives Boolean output and also acts as the destination for geometric products. The Context, Gradient, and Signals regions are each exactly 64 bytes: large enough for one PGA multivector with eight `float64` lanes. Boolean algorithms still access these regions as words, while geometric algorithms reinterpret them as continuous coordinates. The Prev/Next/ID words thread Values into doubly-linked sequences that preserve ingestion order. The Affinity region stores a 257-bit locality-sensitive fingerprint used for content-addressable routing (Section 2.3); the final affinity word uses only its least significant bit.

This layout is not fixed at compile time. A configuration file specifies the word offsets and bit widths for every region, allowing the frame geometry to be altered without modifying the runtime.

The Sixteen Boolean Operations. The Boolean sub-instruction set consists of the sixteen two-input Boolean functions, each encoded as a four-bit truth table. During execution, pairs of bits drawn from two operand spans are used to index into the truth table; the resulting bit is written to the destination. Because these operations are applied across *spans*—contiguous runs of bits within a frame—each instruction performs parallel Boolean evaluation over an arbitrary bit-width in a single step.

The sixteen operations partition into functional roles:

- **Identity and projection** (`A=0011`, `B=0101`): output one operand, ignoring the other. Used for data movement between regions.
- **Equality testing** (XNOR, `1001`): returns all ones where source and destination bits match. A span-wide XNOR producing all ones indicates a perfect match—the basis for pattern recognition across data regions.
- **Difference detection** (XOR, `0110`): returns ones where bits differ. Identifies the structural delta between two frames.
- **Masking and gating** (AND, `0001`; A-AND-NOT-B, `0010`): retains bits in the destination only where the source permits. The inhibitor gate (`0010`) selectively removes bits, functioning as a permission filter.
- **Merging** (OR, `0111`): combines two spans, preserving all active bits from both. Used for superposition of multiple identities into a single frame.
- **Constraint solving** (NAND, `1110`; NOR, `1000`): NAND yields positions where at least one input is inactive—the complement of the AND mask.
- **Conditional selection** (material implication, `1011`, `1101`): retains the destination where the source is active, and forces ones elsewhere.
- **Negation** (NOT-A, `1100`; NOT-B, `1010`): inverts a span.
- **Constants** (`0000`, `1111`): force a span to all zeros or all ones regardless of current content. Used for clearing and signaling.

Geometric ALU Extensions. The Boolean instruction set occupies the low opcode nibble. The high nibble is reserved for geometric opcodes over the 64-byte multivector lanes:

Table 2: Geometric opcode contract. The multivector layout is the even subalgebra of $\mathrm{Cl}(3,0,1)$: $(1, e_{01}, e_{02}, e_{03}, e_{12}, e_{31}, e_{23}, e_{0123})$.

Opcode	Operation	Frame contract
`0x10`	Compose	Signals $\leftarrow$ Context $\cdot$ Gradient
`0x20`	Sandwich	Signals $\leftarrow$ Context $\cdot$ Gradient $\cdot$ Context†
`0x30`	Reverse	Signals $\leftarrow$ Context†

These opcodes add continuous rigid-motion operators without changing the frame stride or removing the Boolean path. The fast affinity filter continues to use XOR and population count; the geometric lane is applied after pruning to rescore candidates, align domains, or re-align episodic memory. Thus higher-level behavior is constructed by composing Boolean and geometric operations across the same in-frame regions.

Program Execution. For Boolean opcodes, the ALU partitions the Token region into two operand spans, A and B. Span A is held steady; span B is rotated in 8-bit steps. The program counter advances through the Program region one opcode at a time, applying the corresponding truth table between A and the rotated B. Results are emitted as Signal bits into the Signals region. For geometric opcodes, the backend reads Context and Gradient as multivectors and writes the 8-lane result into Signals. Programs may set loop bits to request re-enqueuing for iterative execution.

Instruction Encoding. Each instruction occupies a thirty-two-bit word with three fields: a source operand, a destination operand, and an opcode. Boolean firmware uses the low four bits as a truth-table literal. Geometric firmware uses the high nibble to select the PGA operation while preserving the same packed instruction format. Operands may reference named regions, pointer dereferences, or fourteen-bit immediate values. A flag field distinguishes register references from immediates. An assembler translates a textual representation—one instruction per line—into the packed binary form. Extended instruction words support memory-load markers for use in query and prompt firmware.

Programs as Data. Programs reside within the same frame that holds data. The Program region is addressable by the same boolean operations that act on the Token region. A frame may therefore carry stored information, an executable program, or both. Self-modifying behavior is a natural consequence: an instruction can target another instruction in the same program region.

Firmware. The system provides a library of pre-compiled programs called *firmware*. Seven firmware types are defined: **learn** (plasticity operations), **bootloader** (register initialization), **tombstone** (metadata zeroing via FALSE across structural words), **viral** (program propagation to a partner frame), **build** (structural assembly), **query** (retrieval with extended memory-load instructions), and **prompt** (input preparation). Firmware segments compose into multi-stage pipelines through a chaining mechanism described in Section 2.5.

1.2 Computational Substrate

Values do not execute in isolation. The runtime organizes them into a four-layer processing pipeline: the compute backend executes Boolean and geometric programs on individual frames, the MarkovTrie learns from sequences of frames, the Kadabra DHT routes frames to specialized tries, and the Field projects collective pressure across the network.

Multi-Substrate Backend. A load-balanced *Backend* abstracts the heterogeneous hardware available on the host. At initialization, the Backend probes for NVIDIA CUDA devices, Apple Metal devices, and the always-present CPU, registering each as a compute substrate. All substrates implement the same `Execute(frames []unsafe.Pointer)` contract: the programmer compiles intent into the Value layout, the substrate reads the opcode from the Program region, and the backend dispatches internally to the right hardware path.

The Backend selects a substrate for each batch through latency-aware routing:

1. Probe the in-flight depth (number of pending jobs) on each substrate.
2. Route the batch to the substrate with the lowest exponential moving average of service time.
3. When all substrates are saturated, overflow to the CPU.

The dispatch table contains three hot paths. Low-nibble Boolean opcodes use the truth-table ALU. Opcode `0x6` with an in-frame batch marker uses a fused XOR+population-count nearest-affinity kernel over SIMD-padded candidate rows. High-nibble geometric opcodes (`0x10`, `0x20`, `0x30`) use the PGA lane, reading Context and Gradient as eight-lane multivectors and writing the result to Signals.

CPU Boolean execution uses word-parallel logic. The truth table for each instruction is applied via a four-term mask expansion:

$$\text{out} = (A \,\&\, B \,\&\, m_0) \mid (A \,\&\, \bar{B} \,\&\, m_1) \mid (\bar{A} \,\&\, B \,\&\, m_2) \mid (\bar{A} \,\&\, \bar{B} \,\&\, m_3) \tag{1}$$

where $m_0 m_1 m_2 m_3$ are the four bits of the opcode. This executes in constant time per 64-bit word, independent of which of the sixteen Boolean functions is selected. CPU affinity distance uses native SIMD popcount paths, and CPU geometric execution uses hand-written ARM64

and AMD64 floating-point/SIMD assembly for the multivector product. CUDA executes the geometric lane in native `float64`; Metal preserves the 64-bit frame ABI and converts to native `float32` arithmetic inside the shader because Apple Metal does not expose double precision in portable compute shaders.

Priority Spill Ring. A bounded multi-producer, multi-consumer ring buffer (capacity 65,536 slots) provides backpressure-free overflow storage for Values awaiting compute. The ring uses a lock-free enqueue/dequeue algorithm, ensuring that no frame is dropped or blocked during concurrent access.

Machine. The *Machine* is the top-level runtime orchestrator. It binds together the Kadabra node, a dataset source, and the ingestion loop. During operation, the Machine reads the dataset in 64-byte chunks, creates a Value for each chunk, computes the SimHash affinity (Section 2.3.2), links the Value into a doubly-linked sequence chain (setting Prev ID and Next ID), and publishes the Value to the Kadabra DHT for affinity-based routing to the appropriate MarkovTrie. After full dataset ingestion, the Machine triggers training on the accumulated sequences.

Network Transport. A unified connection type, *UniConn*, abstracts three transport mechanisms: inter-process communication (IPC) via Unix domain sockets, UDP multicast for local-network gossip, and QUIC for reliable wide-area transport. All three implement `io.ReadWriteCloser`, allowing Values and FieldDigests to flow across process and network boundaries using the same interface. QUIC provides congestion control and encryption for wide-area deployments; UDP provides low-latency local broadcast for gossip protocol dissemination.

2 Architecture

2.1 Address and Logic Planes

The current architecture separates two concerns without splitting the representation. The same 1024-byte Value frame participates in both the address plane and the logic plane; the distinction is operational rather than structural.

2.1.1 The Address Plane

The address plane is the persistent content-routing layer. Each Value derives a 257-bit affinity vector from its token slab and Kadabra routes the frame to the closest MarkovTrie replicas in that affinity space. This plane is intentionally discrete: XOR distance and population count provide the low-latency filter that reduces a large distributed memory to a small candidate set.

2.1.2 The Logic Plane

The logic plane is the executable in-frame substrate. Low-nibble opcodes apply Boolean truth tables to token, signal, context, and metadata words. High-nibble opcodes interpret Context, Gradient, and Signals as 64-byte PGA multivectors and execute composition, sandwich application, or reversal. The GF(257), GF(8191), and GF(65537) phase hierarchy remains the discrete resonance channel, while PGA transition motors provide the continuous spatial channel used for candidate rescoring, multimodal alignment, and episodic re-alignment.

2.2 Representational Capacity

This subsection analyzes the state space of the Value representation, the information-theoretic properties of the affinity projection, and the capacity amplification provided by the four-layer architecture.

2.2.1 State Space of a Single Value

A Value occupies $D = 8192$ bits (128 words $\times$ 64 bits) and admits 2^{8192} possible binary configurations. Partitioning into functional regions yields the following independent state counts:

$$|\Omega_{\text{token}}| = 2^{1024} \quad \text{(words 0–15)} \tag{2}$$
$$|\Omega_{\text{program}}| = 2^{512} \quad \text{(words 16–23)} \tag{3}$$
$$|\Omega_{\text{signal}}| = 2^{512} \quad \text{(words 24–31)} \tag{4}$$
$$|\Omega_{\text{context}}| = 2^{512} \quad \text{(words 32–39)} \tag{5}$$
$$|\Omega_{\text{gradient}}| = 2^{512} \quad \text{(words 40–47)} \tag{6}$$
$$|\Omega_{\text{meta}}| = 2^{512} \quad \text{(words 48–55)} \tag{7}$$
$$|\Omega_{\text{affinity}}| = 2^{257} \quad \text{(words 123–127)} \tag{8}$$

Because program and data regions are independently addressable, the joint state space of a single frame is the Cartesian product of its regions, substantially larger than any single region alone. The Context, Gradient, and Signals regions also admit continuous interpretations as three eight-lane `float64` multivectors, but their storage remains the same 512-bit lanes accounted for above.

2.2.2 SimHash Projection and Collision Bounds

The affinity region is computed from the token region via five independent SimHash projections (Section 2.3). Each projection h_j ($j = 1, \ldots, 5$) uses a distinct prime stride $p_j \in \{73, 97, 113, 131, 151\}$ to sample $k = 7$ token bits per output bit via majority vote.

For two Values u, v with token Hamming distance $\delta(u, v)$ over the configured token width D_T (1024 bits by default), the per-bit agreement probability of a single SimHash projection is:

$$\Pr\big[h_j(u)_i = h_j(v)_i\big] = \sum_{t=0}^{\lfloor k/2 \rfloor} \binom{k}{t} p^t \, (1-p)^{k-t} \tag{9}$$

where $p = \delta(u, v)/D_T$ is the fractional Hamming distance. At $k = 7$, this function amplifies the similarity signal: inputs differing in fewer than 5% of token bits produce affinity vectors with expected Hamming agreement exceeding 96%, while inputs differing in more than 40% of bits produce near-chance agreement ($\approx 50\%$).

The five projections avoid structural aliasing because $\gcd(p_j, D_T) = 1$ for the default power-of-two token width, and any two projections with strides p_j, p_k first re-collide after $\mathrm{lcm}(p_j, p_k) = p_j \cdot p_k$ steps.

2.2.3 Kademlia Metric from Affinity

The affinity vector defines a metric space via XOR distance. For two Values with affinity words $\mathbf{a}, \mathbf{b} \in \{0,1\}^{257}$, the routing distance is:

$$d(\mathbf{a}, \mathbf{b}) = \mathbf{a} \oplus \mathbf{b} \tag{10}$$

where $\oplus$ denotes bitwise XOR. This satisfies the requirements of a Kademlia topology: $d(\mathbf{a}, \mathbf{a}) = 0$, symmetry $d(\mathbf{a}, \mathbf{b}) = d(\mathbf{b}, \mathbf{a})$, and the triangle inequality (since XOR distance is a metric on $\{0,1\}^n$). The log-distance $\lfloor \log_2 d(\mathbf{a}, \mathbf{b}) \rfloor$ determines bucket placement in the routing table (Section 2.6).

2.2.4 Capacity Amplification Through the Layer Stack

The four-layer architecture multiplicatively expands the accessible state space beyond the single-Value bound:

Layer 1: Value. $|\Omega_1| = 2^{8192}$ per frame.

Layer 2: MarkovTrie. Each MarkovTrie node stores per-label decayed counts and structural metadata. For a trie with M nodes, vocabulary $|V|$, and L labels, the trie state space is $\left(\mathbb{R}_{\geq 0}^{L}\right)^{M} \times |V|^{M}$, growing with the complexity of observed sequences rather than with a fixed parameter budget.

Layer 3: Kadabra DHT. For a network of N trie nodes, the routing topology admits up to $2^{\binom{N}{2}}$ possible edge configurations, constrained by affinity similarity. With replication factor $r = 3$, each Value is stored on r nodes, providing redundancy and enabling multiple tries to specialize on overlapping content regions.

Layer 4: The Field. The eigenmode structure partitions N nodes into K modes. The number of possible partitions is the Bell number B_N. Each partition induces a distinct attention pattern—a set of asymmetric decay and learning multipliers—yielding a total field state space of $B_N \times \mathbb{R}^{2N}$ (one decay and one learning multiplier per node).

2.2.5 Comparison with Parameter-Count Scaling

Table 3: Memory footprint and representational state space. The Value architecture achieves comparable information density per byte to parameterized models, but navigates state space through boolean composition and adaptive trie learning rather than gradient descent.

Architecture	Memory	State Space	$\log_{10}$ States/KB
8192-dim float32 embedding	32 KB	$\sim 2^{262,144}$ (continuous)	$\sim$ 2,467
GPT-2 (117M params, float16)	234 MB	$\sim 2^{1.87\times10^9}$	$\sim$ 2,406
1K Values (frames only)	1 MB	$(2^{8192})^{1000}$	$\sim$ 2,467
1K Values + MarkovTrie + Field	1 MB + trie	$(2^{8192})^{1000} \times \lvert\Omega_{\text{trie}}\rvert$	$>$ 2,467

The states-per-kilobyte ratio is comparable across architectures, as information density is ultimately bounded by the entropy of the underlying bits. The key difference is the navigation strategy: parameterized models traverse state space through gradient updates over continuous parameters, while the Value architecture traverses it through Boolean composition and localized PGA transformations at the frame level, surprise-modulated trie updates at the learning level, and eigenmode selection at the collective level (Section 2.4).

2.3 Affinity Projection and Topological Routing

2.3.1 The Shannon Limit and Bundling Saturation

A persistent challenge in Hyperdimensional Computing (HDC) and Vector Symbolic Architectures (VSA) is the capacity bound imposed by bundling saturation. When distinct concepts are overlaid into the same D-dimensional binary vector via bitwise OR, the population density of active bits monotonically increases. Representational capacity is maximized when exactly 50% of bits are active ($p(1) = 0.5$); beyond this threshold, structural uniqueness decays and proximity searches yield pervasive false positives.

Conventional architectures delay this saturation by scaling the dimensionality. The approach taken here is different: rather than expanding the vector, the system embeds a *locality-sensitive affinity fingerprint* directly into the frame and uses it to route Values to specialized storage nodes, distributing the representational load across a network of independent tries.

2.3.2 SimHash Affinity Projection

The affinity region of each Value (words 123–127, 257 meaningful bits) is computed from the token region (words 0–15, 1024 bits in the default configuration) via five independent SimHash-style projections. The first four projections write full 64-bit words; the fifth writes the single remaining bit, yielding four full words plus one masked bit. Each projection h_j ($j = 0, \ldots, 4$) uses a distinct prime stride p_j to sample the token bits:

Definition 1 (SimHash Projection)**.** *Let D_T be the configured token-bit count and let $b_j = 64$ for $j < 4$ and $b_4 = 1$. For output bit $i \in \{0, \ldots, b_j - 1\}$ of projection j, sample $k = 7$ token bits at positions:*

$$idx(i, j, s) = (i \cdot p_j + s \cdot p_j + 37j) \bmod D_T \qquad s = 0, \ldots, k-1 \tag{11}$$

The output bit is set if a majority of the k sampled bits are set (majority vote):

$$h_j(x)_i = \mathbb{1}\Big[\sum_{s=0}^{k-1} x_{idx(i,j,s)} \geq \lceil k/2 \rceil\Big] \tag{12}$$

The prime strides $\{73, 97, 113, 131, 151\}$ are chosen so that, for the default power-of-two token span, $\gcd(p_j, D_T) = 1$ for all j. Distinct prime strides avoid sub-lattice aliasing and delay realignment: any two projections first re-collide after $\text{lcm}(p_j, p_k) = p_j \cdot p_k$ steps.

The majority vote with $k = 7$ has error rate $\sim \exp(-2 \cdot \text{margin}^2 \cdot k)$, producing $> 96\%$ per-bit accuracy for true-positive inputs. At $k = 7$, each output bit samples $\approx 1.4\%$ of the input, preserving sensitivity to $\sim 5\%$ input differences while suppressing single-bit noise.

2.3.3 Bloom Filter Fallback

For low-entropy inputs where the majority-vote LSH produces an all-zero affinity vector, the system forces a single distinguishing bit so Kadabra routing never receives a degenerate key. When raw context bytes are available, the runtime instead ORs FNV-1a hashes of overlapping 4-byte and 8-byte n-grams into the same 257-bit affinity region. This path captures substring overlap more directly than a majority vote over a sparse token slab and still obeys the final-word mask.

2.3.4 Topological Affinity Clustering

Values are not stored uniformly. The affinity vector determines which node in the Kadabra DHT (Section 2.6) receives a Value. Topological proximity between two Values u, v is measured by:

$$\text{overlap}(u, v) = \sum_{w=0}^{4} \text{popcount}(\mathbf{a}_{u,w} \ \& \ \mathbf{a}_{v,w}) \tag{13}$$

where word four is masked to one meaningful bit. Field-level coupling uses the related Jaccard score

$$C_{\text{aff}}(u, v) = \frac{\text{popcount}(\mathbf{a}_u \,\&\, \mathbf{a}_v)}{\text{popcount}(\mathbf{a}_u \mid \mathbf{a}_v)} \tag{14}$$

with zero returned for the empty-union case. Values with high overlap are routed to the same network node, creating implicit clusters of semantically related content. This mechanism replaces external routing heuristics: the data itself dictates its storage location through the affinity fingerprint embedded in its frame.

2.3.5 State Space Analysis

The affinity mechanism amplifies representational capacity through topological structuring rather than dimensional scaling.

Level 1: The 8192-Bit Value. The atomic unit admits $|\Omega_{\text{frame}}| = 2^{8192}$ binary configurations.

Level 2: The Distributed Topological Graph. For N active Values, the affinity-routed topology admits up to $(2^{8192})^N \times \mathcal{O}(2^{N(N-1)/2})$ possible configurations—the product of individual frame states and the space of possible routing edges.

Level 3: Trie and Field States. Each Value's storage node maintains a MarkovTrie (Section 2.7) whose state grows with observed complexity, and the Field (Section 2.4) partitions nodes into eigenmodes whose multipliers modulate learning and decay. The total accessible state space is the product of all three levels, growing multiplicatively without requiring dimensional expansion of individual frames.

2.4 The Field: Emergent Attention via Eigenmode Selection

The Field is not a data structure that nodes query. It is a *force that acts on them*—analogous to a physical field acting on particles. It emerges from the gossip protocol (Section 2.6.4): as nodes exchange `FieldDigest` messages, each node's `FieldView` detects emergent clusters of structurally aligned, phase-coherent tries, identifies the dominant cluster, and projects asymmetric pressure back onto the local trie. This mechanism functions as the system's attention: it selects what to retain and what to forget, without matrices, softmax, or learned parameters.

2.4.1 Affinity Coupling: Structural Alignment

Structural similarity between two nodes is measured by *affinity coupling*, implemented as Jaccard overlap over the 257-bit affinity vectors. For affinity vectors $\mathbf{a}, \mathbf{b} \in \{0,1\}^{257}$, with the final word masked to one meaningful bit, the coupling is:

$$C_{\text{aff}}(\mathbf{a}, \mathbf{b}) = \frac{\text{popcount}(\mathbf{a} \,\&\, \mathbf{b})}{\text{popcount}(\mathbf{a} \mid \mathbf{b})} \tag{15}$$

and is defined as zero when the union is empty. The coupling threshold for eigenmode membership is not a fixed constant; the Field feeds all pairwise coupling values through an exponential moving average and uses the resulting local estimate as θ_C. This keeps the gate tied to the observed routing distribution rather than to a hand-selected absolute overlap.

2.4.2 Phase Coherence: Surprisal Velocity Alignment

Structural alignment alone is insufficient for mode membership. Two nodes may have similar affinity vectors but be in complementary learning states (one absorbing novelty while the other consolidates). Phase coherence captures this dynamic alignment by comparing *surprisal velocities*:

$$v_i = \bar{S}_i^{(t)} - \bar{S}_i^{(t-1)} \tag{16}$$

where $\bar{S}_i^{(t)}$ is node i's mean surprisal at epoch t. The phase coupling between two nodes is:

$$C_{\text{phase}}(i,j) = \frac{v_i \cdot v_j}{\sqrt{|v_i|} \cdot \sqrt{|v_j|} + \epsilon} \tag{17}$$

with $\epsilon = 0.01$ to suppress spurious coupling when both nodes are quiescent. This is a normalized product: $C_{\text{phase}} = +1$ when both nodes are accelerating or both decelerating (in-phase), $C_{\text{phase}} = -1$ when one is rising and the other falling (anti-phase), and $C_{\text{phase}} \approx 0$ when at least one node is stationary.

The geometric-mean normalization $\sqrt{|v_i| \cdot |v_j|}$ is used instead of $\max(|v_i|, |v_j|)^2$ to avoid over-penalizing asymmetric velocity pairs—a common case when one node has just begun learning while a structurally aligned neighbor is in full swing.

The current implementation uses phase coherence during pressure projection, not as a hard clustering gate. Coupled neighbors are weighted by $C_{\text{aff}}(\mathbf{a}_i, \mathbf{a}_j) \cdot (1 + C_{\text{phase}}(i,j))$, so co-moving nodes amplify each other's pressure, opposing nodes are damped, and quiescent nodes contribute little.

2.4.3 Eigenmode Detection

Given the set of received digests $\{d_1, \ldots, d_N\}$, the `FieldView` identifies eigenmodes by greedy clustering:

Algorithm 1 Eigenmode Detection

Estimate θ_C from the EMA of all pairwise affinity couplings
$\mathcal{M} \leftarrow \emptyset$ ▷ set of modes
for each digest d_i **do**
  if d_i is already assigned **then**
    continue
  end if
  Create new mode $m = \{d_i\}$ with $E(m) = \bar{S}_i$
  for each unassigned digest d_j **do**
    if $C_{\text{aff}}(\mathbf{a}_i, \mathbf{a}_j) \geq \theta_C$ **then**
      Add d_j to m and set $E(m) \leftarrow E(m) + \bar{S}_j$
    end if
  end for
  $\mathcal{M} \leftarrow \mathcal{M} \cup \{m\}$
end for
Dominant mode: $m_{\text{dom}} = \arg\max_{m \in \mathcal{M}} E(m)$

The energy of a mode $E(m) = \sum_{i \in m} \bar{S}_i$ aggregates the absolute surprisal mass of its members. The dominant mode—the one with the highest total energy—is what the system "attends to." The greedy implementation deliberately avoids maintaining mutable centroids: each unassigned digest acts as a seed, and later unassigned digests join that mode when their affinity coupling to the seed clears the adaptive threshold.

2.4.4 Asymmetric Field Projection

Once the dominant mode is identified, the `FieldView` projects asymmetric pressure onto the owning node's MarkovTrie. The asymmetry is derived from the current mode energy distribution, not from fixed constants. Let

$$r = \max\left(\frac{E(m_{\text{dom}})}{\sum_m E(m)}, \epsilon\right) \tag{18}$$

where the ratio is smoothed through the adaptive chain and ϵ prevents reciprocal blow-up. The base multipliers are:

Table 4: Field pressure multipliers. The ratio is derived from the dominant-mode energy share and is further modulated by mesh-wide phase alignment.

	Aligned (in dominant mode)	**Misaligned (outside)**
Decay multiplier	r^{-1}	r
Learning multiplier	r	r^{-1}

These multipliers modulate the base field pressure, which is itself derived from the weighted average of coupled neighbors' surprisal and growth rates. For a node with local surprisal $\bar{S}_{\text{local}}$ and field-averaged surprisal $\bar{S}_{\text{field}}$, the raw pressure differentials are:

$$\Delta_{\text{decay}} = \bar{S}_{\text{field}} - \bar{S}_{\text{local}} \tag{19}$$

$$\Delta_{\text{learn}} = \dot{N}_{\text{field}} - \dot{N}_{\text{local}} \tag{20}$$

where the field averages are weighted by the product of affinity coupling and phase coupling: $w_{ij} = C_{\text{aff}}(\mathbf{a}_i, \mathbf{a}_j) \cdot (1 + C_{\text{phase}}(i, j))$.

The raw pressures are multiplied by the alignment-dependent factors from Table 4, then modulated by the current global GF(65537) phase. If the trie-local GF(257) mode aligns with

the global mode, learning is boosted by $1 + ca$ and decay is divided by the same factor, where c is global phase concentration and a is circular phase alignment. Misaligned tries receive the complementary decay boost $1 + c(1 - a)$ and have learning scaled by the residual alignment floor. Both decay and learning are clamped by adaptive spread estimates rather than by fixed numeric bounds.

2.4.5 Interpretation as Attention

The Field mechanism implements attention without the machinery of query-key-value projections. The "query" is each node's current adaptive state (broadcast as a digest). The "key matching" is affinity coupling plus surprisal-velocity phase coherence. The "value aggregation" is the weighted field pressure from coupled neighbors. The "selection" is the asymmetric fork: amplify the attended mode, suppress the rest.

This produces three effects that are analogous to attention in transformer architectures:

1. **Concentration**: aligned nodes retain knowledge longer (suppressed decay) and absorb related input faster (amplified learning), concentrating capacity on the dominant pattern.
2. **Graceful transition**: because the suppression is multiplicative rather than binary, minority modes are dampened but not destroyed. When the dominant pattern shifts, suppressed nodes still retain enough state to become the new dominant mode.
3. **No catastrophic forgetting**: the multiplier is derived from mode energy and clamped by the adaptive spread estimator, so a temporarily misaligned node is dampened without being assigned an unbounded decay rate.

2.5 Firmware Chaining and Program Composition

The program region of a single Value holds 32 instruction slots. This is not, however, the limit of a program's length. Because the firmware index and program counter are addressable by the same boolean operations used for all other computation, a program can *chain* to the next firmware segment by writing a new firmware index and resetting the program counter as its final instructions. An arbitrarily long computation can therefore be expressed as a sequence of firmware segments linked end-to-end.

2.5.1 Firmware Compilation

Each firmware segment is specified as a sequence of textual instructions in the format `source destination operation`. Operands may name a register, a pointer dereference, or a fourteen-bit immediate value. The operation field is a four-bit binary literal selecting one of the sixteen truth tables for Boolean firmware. Geometric firmware uses the same packed instruction word while setting the high opcode nibble to select the PGA lane. An assembler translates each line into a packed thirty-two-bit instruction word. Extended instruction words (encoded as raw 32-bit integers) support memory-load markers and active-slot directives used by the query and prompt firmware.

2.5.2 The Firmware Library

Seven firmware segments are defined, each performing a focused structural operation:

The **learn** firmware executes plasticity operations that prepare a frame for trie insertion. The **bootloader** initializes registers to known values and prepares a frame for subsequent execution. Every firmware chain begins here: bootloader $\rightarrow$ segment$_1$ $\rightarrow$ segment$_2$ $\rightarrow \cdots$.

The **tombstone** firmware zeroes structural metadata by applying the FALSE operation (truth table `0000`) across link pointers, state fields, and affinity words, marking a frame as inactive. This is used to garbage-collect Values that have been fully absorbed by the trie.

The **viral** firmware copies the executing frame's program region into a partner frame's program region. Its final instructions write a specified firmware index into the partner's firmware register and reset the partner's program counter, so that the partner wakes up running the next segment in the chain.

The **build** firmware assembles structural content by composing boolean operations across multiple frame regions. The **query** firmware performs retrieval by executing extended memory-load instructions that mark active slots for content-based matching against trie contents. The **prompt** firmware prepares input data for query execution, formatting raw bytes into the expected slot layout.

2.5.3 Firmware Chains as Pipelines

The chaining mechanism allows firmware segments to compose into multi-stage pipelines. A retrieval pipeline, for instance, might proceed as:

1. **Bootloader:** initialize registers and prepare the frame.
2. **Prompt:** format the input query into active slots.
3. **Query:** execute memory-load instructions against the trie, matching slot content to stored sequences.
4. **Build:** assemble the retrieved fragments into a coherent response.

Each transition is a two-instruction firmware-index/program-counter handoff. The pipeline length is limited only by the number of defined firmware segments, not by the 32-slot capacity of a single program region.

2.5.4 Self-Modification and Program Discovery

Because the program region is part of the frame's addressable memory, boolean operations applied during trie interaction can produce instruction sequences that differ from any of the firmware templates. If such a derived program produces a useful transformation, the viral firmware can propagate it to neighboring frames.

The selection signal for "useful" arises from the trie's own adaptive statistics (Section 2.7). A frame whose program produces low-surprisal predictions will be retained longer by the decay mechanism; a frame whose program produces high-surprisal outputs will trigger aggressive learning. The viral mechanism preferentially copies programs from frames in the dominant eigenmode (Section 2.4), creating a feedback loop: firmware seeds initial behavior; trie interaction refines programs; the field selects which refinements propagate; and the cycle repeats. No gradient computation is involved.

2.6 Kadabra: Affinity-Routed Distributed Hash Table

The Kadabra layer organizes MarkovTrie instances into a Kademlia-style distributed hash table where nodes *are* tries. Values route to tries based on affinity similarity, creating emergent specialization: each trie accumulates expertise in the content domain defined by its neighborhood in affinity space.

2.6.1 Routing Architecture

Each node in the network is identified by a 64-bit `NodeID` and maintains a routing table of 64 k-buckets (one per bit of the ID space), each holding up to $B = 20$ peer references. Bucket i contains peers whose XOR distance from the local node has its highest set bit at position i:

$$\text{bucket}(a, b) = \lfloor \log_2(a \oplus b) \rfloor \tag{21}$$

This produces the standard Kademlia property that nearby nodes share low-index buckets (many common prefix bits) and distant nodes populate high-index buckets.

2.6.2 Affinity-Based Value Routing

When a Value is published to the network, its 257-bit affinity vector (Section 2.3) determines the target nodes. The system computes the XOR distance between the Value's affinity and each candidate node's affinity, then routes the Value to the $r = 3$ closest nodes (the replication factor). Because the affinity vector is a locality-sensitive hash of the token content (Section 2.2), Values with similar content naturally cluster on the same nodes. This produces *emergent specialization*: each trie receives, and therefore learns from, a coherent subset of the input stream.

2.6.3 Adaptive Peer Selection via Multi-Armed Bandit

Peer quality within each bucket is tracked via two online statistics: latency (exponential moving average of round-trip time) and success rate (fraction of recent queries that returned results). Every $E_{\text{epoch}} = 100$ queries, the system re-evaluates each bucket:

1. Compute a quality score for each peer: $q = \alpha_{\text{success}} \cdot \text{success\_rate} - \alpha_{\text{latency}} \cdot \text{latency\_ema}$.
2. With exploration probability ϵ, probe a random candidate not currently in the bucket.
3. If the candidate's quality exceeds the worst current peer, swap them.

This multi-armed bandit strategy ensures that the routing table continuously improves without requiring global coordination.

2.6.4 Gossip Protocol and Field Digests

At each epoch boundary, every node broadcasts a compact `FieldDigest` to its routing peers. A digest contains:

- The originating `NodeID`.
- The node's 257-bit affinity vector.
- Surprisal mean $\bar{S}$, variance σ_S^2, and previous-epoch mean $\bar{S}_{\text{prev}}$ (for computing phase velocity).
- Classification entropy H.
- Node growth rate $\dot{N}$ (trie node creation rate).
- Effective trie depth d_{eff}.
- Epoch counter.

Digests are lightweight (a few hundred bytes) relative to the trie state they summarize. Each receiving node absorbs the digest into its `FieldView`, which triggers eigenmode recomputation and field projection (Section 2.4). Digests are deduplicated by epoch number: a digest with an epoch $\leq$ the stored epoch for that origin is silently dropped.

2.6.5 Replication and Redundancy

Each Value is stored on $r = 3$ nodes selected by affinity proximity. This serves two purposes: fault tolerance (any single node failure leaves two replicas) and learning diversity (multiple tries learn from the same data, potentially extracting different structural patterns based on their existing knowledge). The replication factor is configurable but defaults to 3, consistent with standard Kademlia deployments.

2.7 The MarkovTrie Store

The MarkovTrie is the system's learning substrate: an adaptive probabilistic suffix trie that learns online from every observation without training epochs, loss functions, or weight matrices. Each node in the Kadabra routing layer (Section 2.6) owns one MarkovTrie instance; Values are routed to tries by affinity similarity, so each trie naturally specializes in a region of content space.

2.7.1 Data Structure

The trie is a rooted tree in which each node n stores:

- A token τ_n (the symbol at this position in the suffix).
- A children map $\mathcal{C}_n : \mathcal{V} \to \mathcal{N}$ from vocabulary symbols to child nodes.
- Per-label decayed counts $c_n(\ell) \in \mathbb{R}_{\geq 0}$ for each label $\ell \in \mathcal{L}$.
- A total visit count $T_n = \sum_\ell c_n(\ell)$.
- Depth d_n (distance from the root).
- A timestamp t_n recording the last update step.
- A transition motor M_n derived from the parent and child Context multivectors.

Paths from root to leaf encode observed token sequences. The depth of the trie grows with the complexity of the data, not with a fixed parameter budget.

2.7.2 Lazy Decay

All counts decay exponentially, but decay is computed lazily: a node's counts are updated only when it is accessed. If the current global step is s and a node was last touched at step t_n, the elapsed decay is:

$$c_n(\ell) \leftarrow c_n(\ell) \cdot \lambda^{s - t_n} \tag{22}$$

where $\lambda \in (0, 1)$ is the decay factor (default 0.995). This ensures that stale knowledge fades without requiring a global sweep, and the cost is amortized over accesses.

2.7.3 Surprisal-Modulated Plasticity

The trie's learning rate is not a fixed hyperparameter. It is modulated by *surprisal*: the information-theoretic novelty of each observation. For a token τ observed in context c, the surprisal is:

$$S(\tau \mid c) = -\log_2 P(\tau \mid c) \tag{23}$$

where $P(\tau \mid c)$ is the trie's current estimate of the next-token probability given context c. The effective learning rate for this observation is:

$$\eta(\tau) = \min\bigl(\eta_{\max},\ \eta_{\text{base}} + \bar{S}/\sigma_S\bigr) \tag{24}$$

where $\bar{S}$ is the exponential moving average of recent surprisal values, σ_S is its standard deviation, and $\eta_{\text{base}} = 0.1$, $\eta_{\max} = 1.0$ are the floor and ceiling. Novel inputs (high surprisal) are learned aggressively; familiar inputs (low surprisal) are absorbed gently.

2.7.4 Adaptive Self-Tuning

Nine parameters that are typically fixed hyperparameters are instead derived online from signals the trie already computes. All use exponential moving averages with smoothing factor $\alpha = 0.05$ (effective window ≈ 20 observations). The adaptive mappings are:

A cold-start guard prevents any adaptive parameter from deviating from its base value until at least 50 surprisal observations have been collected, ensuring that the EMA estimates are calibrated before they influence behavior.

2.7.5 Classification

Classification follows a Naive Bayes formulation over interpolated n-gram probabilities. For an input sequence $\mathbf{x} = (x_1, \ldots, x_T)$ and label ℓ, the log-evidence is:

$$\log P(\ell \mid \mathbf{x}) \propto \log P(\ell) + \sum_{t=1}^{T} \sum_{d=1}^{D} w_d \log P_d(x_t \mid x_{t-d:t-1},\, \ell) \tag{25}$$

Table 5: Adaptive parameter derivation. Each parameter is a deterministic function of an online signal; no search or meta-optimization is required.

Parameter	**Signal**	**Mechanism**
Decay factor λ	Surprisal EMA	High surprisal $\rightarrow$ lower λ (forget faster)
Learning rate η	Per-token surprisal	Novel tokens learned more aggressively
Prune threshold	Node growth rate	Fast growth $\rightarrow$ prune harder to control memory
Classification context	Classification entropy	High entropy $\rightarrow$ widen context window
Temperature τ	Surprisal EMA	Familiar $\rightarrow$ explore; novel $\rightarrow$ exploit
Beam width	Classification confidence	Low confidence $\rightarrow$ wider search
Interpolation weights	Depth hit rate	Tracks which n-gram depths are predictive
Episodic blend weight	Episodic match quality	Good episodic hits $\rightarrow$ higher blend
Unsupervised threshold	Classification accuracy	Maturing labels $\rightarrow$ require more confidence

where P_d is the d-gram probability read from the trie at depth d, w_d is the adaptive interpolation weight for depth d, D is the interpolation suffix depth, and $P(\ell)$ is the label prior from global class totals. The system walks the trie depth-first, accumulating log-evidence per label, and returns the label with the highest posterior.

2.7.6 Generation

Text generation uses beam search with adaptively derived parameters. At each step, the trie samples candidate next tokens from the distribution at the current node:

$$P(\tau \mid c) = \frac{c_n(\tau)}{\sum_{\tau'} c_n(\tau')} \tag{26}$$

Temperature τ, beam width, and maximum length are not hyperparameters; they are derived from the trie's adaptive state (Table 5). Beam candidates are extended while their cumulative probability exceeds a threshold, producing completions whose length and diversity adapt to the trie's confidence in the current context. Candidate rescoring is two-phase. First, the trie-local GF(257) phase vector boosts continuations that interfere constructively with the current local phase. Second, the geometric lane derives a candidate motor from the current Context multivector and the candidate's deterministic multivector, applies it through the sandwich product, and boosts candidates whose steered point aligns with the local attractor:

$$\text{boost}(\tau) = \log\left(1 + g \cdot \frac{\cos(M_\tau X M_\tau^\dagger, A) + \cos(X_\tau, A)}{2}\right) \tag{27}$$

where X is the current context coordinate, X_τ is the candidate coordinate, A is the mean Context multivector over the prediction path, and g is a fixed gain in the scorer.

2.7.7 Cross-Domain Procrustes Alignment

The `MultimodalCoordinator` maintains independent sensory, action, and reward tries. When sensory–action–reward triples are observed, the coordinator records immutable coactivation statistics: terminal affinities, terminal IDs, reward residuals, and the sensory, action, and reward Context multivectors. Once at least eight paired sensory/action samples are available, it solves an Orthogonal Procrustes problem:

$$R^* = \arg\min_{R^\mathsf{T} R = I} \|AR - B\|_F \tag{28}$$

where A contains sensory coordinates and B contains action coordinates. The resulting rotation is published atomically as an immutable domain alignment. During policy inference, sensory coordinates can be projected into the action manifold before candidate actions are scored, giving the coordinator a direct geometric path in addition to affinity and causal-residual evidence.

2.7.8 Episodic Memory

A bounded episodic buffer (default capacity 1000) stores recent token sequences with timestamps and a deterministic geometric coordinate for each event. During prediction, the buffer contributes a k-nearest-neighbor tail to the trie's probability estimates. The blend weight between trie predictions and episodic recall is itself adaptive: when episodic matches are high-quality (low edit distance to the query), the blend weight increases; when matches are poor, the trie's own estimates dominate. This provides fast adaptation to recent context without retraining the trie.

The buffer also exposes a Procrustes re-alignment operation for idle consolidation. A caller samples stored events, resolves their current coordinates in the backing trie, solves a Procrustes rotation from old episodic coordinates to current trie coordinates, and applies that rotation to the whole buffer. This prevents old short-term memories from drifting away from a manifold that has moved under continued learning.

2.7.9 Experience-Driven Learning

The trie does not distinguish training from inference. When the `Predict` method encounters high-surprisal input (average per-token surprisal above a configurable threshold, default 1.0 bit), it invokes `Experience` before classification. Experience creates a new concept label if no existing label adequately explains the input, inserts the sequence into the trie under that label, and updates all adaptive statistics. This closed loop means the trie continuously refines its model as it processes queries.

2.8 Algebraic Structure and Group-Equivariant Computation

The Geometric Deep Learning programme formalized by Bronstein et al. (**?**) demonstrates that many successful neural architectures—CNNs, GNNs, Transformers—can be derived as special cases of equivariant message-passing on domains with specific symmetry groups. Cohen and Welling's work on Group Equivariant Convolutional Networks (**?**) provides the formal justification: when the underlying data possesses a known symmetry, restricting transformations to that symmetry group improves sample efficiency and generalization.

The Value architecture relates to this programme through two algebraic lanes. The low-nibble Boolean instruction set forms a closed algebraic structure: composing any two operations yields another operation expressible as a sequence of truth-table applications. When the same instruction is applied uniformly across all bit positions of a frame, the operation is translation-equivariant in the bit-position coordinate— shifting the input by k positions and then applying the operation produces the same result as applying the operation and then shifting. The high-nibble PGA lane adds a continuous group-action channel for rigid transitions and reversals.

This equivariance is less expressive than learned group-equivariant networks, which can adapt their feature maps to continuous symmetries of the input domain. The Value architecture instead provides fixed algebraic structures—Boolean algebra over $\{0,1\}^{8192}$ and a small PGA multivector algebra over selected 64-byte lanes—that are deterministic and algebraically closed by construction. The trade-off is rigidity: the system does not learn arbitrary new symmetries from data, but the symmetries it does possess are guaranteed rather than approximate, and they can be exploited at the hardware level.

Non-Commutativity and Sequence Sensitivity. Although individual boolean operations are bitwise-commutative (AND is commutative), sequences of operations applied to a frame are generally *non-commutative*: applying operation A followed by operation B to a frame typically produces a different result than applying B followed by A. This property allows the order in which frames interact during Region mixing to encode structural information. Two sequences of the same frames, mixed in different orders, may yield different final states. In this sense, the non-commutativity of composed Boolean and geometric operations serves a role analogous to positional encoding in sequence models, without requiring a learned position representation.

2.9 Deterministic Encoding and Collision Properties

The system encodes input bytes into Value frames through a deterministic mapping that assigns each byte sequence a fixed binary representation. This subsection places this encoding in the context of two classical results: Gödel numbering and arithmetic coding.

Relationship to Gödel Numbering. Gödel's 1931 encoding assigns every formula of a formal language a unique natural number via products of prime powers. The Fundamental Theorem of Arithmetic guarantees that this mapping is injective: distinct formulas receive distinct numbers, and the original formula can be recovered from its number. The Value encoding shares the structural goal of providing a deterministic, reproducible mapping from symbolic input to a fixed binary representation, though it operates at the byte level rather than over formal language symbols.

The key property inherited from the Gödel construction is *determinism*: the same input byte sequence always produces the same Value content, and distinct inputs that differ in at least one byte position produce Values that differ in the corresponding data-region words. This guarantee holds without learning or randomized hashing; it is a consequence of the encoding rule itself.

Collision and Deduplication. When two distinct input sequences produce Values that share a common prefix in their data regions, the shared structure can be exploited for storage efficiency. This is analogous to the principle underlying arithmetic coding (**?**), in which symbols are mapped to sub-intervals of $[0, 1)$ proportional to their empirical probability, achieving entropy-rate compression.

In the Value architecture, deduplication arises naturally when multiple inputs map to identical data-region contents. Because the mapping is deterministic, identical byte sequences always produce identical Values, and a storage layer that indexes Values by their content can coalesce duplicates without additional computation. The degree of deduplication depends on the redundancy structure of the input data: highly repetitive corpora (such as natural language, where Zipf's law governs token frequencies) produce more collisions than uniformly random input.

Implications for Storage Scaling. This collision property means that the effective memory footprint of a collection of Values scales with the information content of the data rather than its raw volume. The compression ratio is bounded below by the Shannon entropy of the input distribution—the same theoretical floor that governs arithmetic coding. Empirical measurements of deduplication rates across different data domains are reported in the scaling experiments.

2.10 Hardware-Accelerated Frame Computation

The Value architecture is designed to align with the native instruction sets of modern parallel hardware. The fast filter path reduces to bitwise logic (AND, OR, XOR, NOT), population count, and integer comparison—instructions that execute efficiently on contemporary CPU, GPU, and accelerator architectures. The current implementation also adds a bounded geometric path for PGA multivector products after the Boolean candidate set has been pruned.

SIMD and GPU Mapping. Each Value is 1024 bytes. Boolean truth-table execution and affinity distance kernels operate directly over the 128 64-bit words of the frame, while the geometric lane reads three 64-byte regions as eight-lane multivectors. On GPU hardware, the Boolean path maps to threadgroup dispatch over words or candidate rows, and the geometric path maps to one frame-local multivector product per work item.

The system provides three backend implementations:

- **CPU:** A word-parallel Boolean implementation, native SIMD affinity distance kernels, and hand-written ARM64/AMD64 assembly for the PGA geometric product.
- **Apple Metal:** Compute shaders dispatch value operations across threadgroups. The geometric lane preserves the 64-bit frame ABI and converts to native `float32` arithmetic within the shader.
- **NVIDIA CUDA:** Equivalent kernels dispatch across warps, with coalesced memory access patterns and native `float64` arithmetic for the geometric lane.

Deterministic Latency. Because every instruction is a fixed-length operation over a fixed-size frame, execution time is tightly bounded: there are no variable-length encodings and no data-dependent loops. The Boolean path remains the lowest-latency path and supplies the first-stage filter. The geometric path introduces floating-point arithmetic, so its precision properties are substrate-specific: CUDA and CPU use `float64`, while Metal uses `float32` inside the shader while preserving the frame's 64-bit storage contract.

Write-Optimized Access Patterns. The backend tracks per-substrate pressure with atomic counters and an exponential moving average of service time. Residency metadata in the frame lets the router penalize unnecessary physical hops, while periodic exploration prevents the scheduler from sticking to a stale substrate choice. Overflow work is admitted through the pool queue rather than a blocking synchronization point.

Relationship to In-Memory Computing. The Boolean portion of Value execution is particularly well-suited to emerging in-memory computing architectures, where bitwise operations can be performed directly within memory arrays without transferring data to a separate processing unit. The geometric portion is intentionally bounded to three 64-byte regions, making it a separable acceleration target for substrates that provide efficient vector floating-point execution.

2.11 Hyperdimensional Computing and Vector Symbolic Architectures

The Value representation is situated within the framework of *Hyperdimensional Computing* (HDC) and *Vector Symbolic Architectures* (VSA). In the canonical VSA formulation (**?**), data is represented as high-dimensional random vectors (typically $D \geq 10{,}000$) and manipulated through three algebraic operations: *binding* (element-wise multiplication or circular convolution), *bundling* (element-wise addition or disjunction), and *permutation* (coordinate rotation). These operations form an approximate algebra over a vector space, enabling compositional representations of structured data without learned parameters.

The Value maps onto this framework with specific design choices. At 8192 bits, each frame operates in a dimensionality approaching the classical HD regime ($D \geq 4{,}000$), gaining the concentration-of-measure benefits that make high-dimensional binary vectors approximately orthogonal under random sampling. The Boolean instruction set provides all sixteen two-input operations, subsuming the standard VSA binding and bundling operators as special cases: bitwise AND serves as a similarity probe, bitwise OR implements bundling through disjunction, and circular shift (achievable through register-offset addressing) provides positional binding. The geometric opcode lane adds a second VSA-like operator family: PGA motors compose transitions and apply them to target coordinates without leaving the fixed frame.

Gayler's analysis (**?**) demonstrated that VSAs support systematic compositional generalization because binding and bundling preserve algebraic structure. The Value architecture inherits this property: the encoding of input bytes into the data region is deterministic, the sixteen Boolean operations are closed over the binary frame, and the geometric lane is closed over the eight-lane multivector representation stored in Context, Gradient, and Signals.

A notable difference from classical HDC is the absence of a fixed bundling threshold. Standard VSAs operate in a flat vector space where repeated bundling via disjunction monotonically increases the population count (the number of active bits), eventually degrading all similarity queries when the count exceeds approximately 50% of the dimensionality. The Value architecture addresses this through the multi-tier Region structure: rather than accumulating all information into a single vector, the system distributes content across a population of frames that interact combinatorially. Saturation is a property of individual frames, not of the system as a whole.

2.12 Manifold Capacity and Representation Geometry

Manifold Capacity Theory. Chung, Lee, and Sompolinsky (**?**) developed a statistical-physics theory of *perceptual manifold capacity*: the maximum number of object manifolds that a linear readout can separate. Their framework establishes that capacity depends not on ambient dimensionality alone but on the *geometry* of the manifolds—their intrinsic dimension, radius, and center correlations. Recent extensions (**?**) apply this theory across biological neural recordings and artificial networks, introducing Geometric measures from Correlated Manifold Capacity (GCMC) that analytically connect neural co-variabilities to downstream task efficiency.

The Value representation provides a concrete setting in which these geometric quantities can be characterized. Each Value occupies a point in $\{0,1\}^{8192}$ and also carries an eight-lane PGA coordinate in its Context region. A collection of Values derived from related inputs (for example, different byte sequences sharing a common prefix) therefore defines both a discrete affinity manifold and a continuous local transition manifold. The relevant geometric properties are:

- **Intrinsic dimension:** bounded by the number of bits that vary across the manifold, which depends on the length and diversity of the input sequences.
- **Radius:** controlled by the Hamming distance between the most dissimilar Values in the manifold.
- **Center correlations:** determined by the shared bit-pattern structure among Values derived from related inputs.

A population-count-based similarity score between two Values—the Hamming inner product—functions as a linear readout over the binary manifold space, while cosine similarity between multivectors functions as the local readout for the geometric lane. The discriminative capacity of these readouts is therefore governed by the geometric properties that Chung's theory characterizes. Formal application of manifold capacity bounds to the Value representation is a direction for future analytical work.

Relationship to Noncommutative Algebra. Hataya and Hashimoto (**?**) provide a formal proof that noncommutative product structures in parameter spaces induce improved feature learning for structured tasks. Their framework generalizes neural network parameters from scalars to elements of a noncommutative C^*-algebra.

As discussed in Section 2.8, the composition of Boolean and PGA operations over Values is non-commutative: different orderings of the same operations produce different frame states. This non-commutativity is structural rather than learned—it arises from the sequential application of truth tables and geometric products to a shared frame substrate. The question of whether this prescribed non-commutative structure yields the representational benefits that Hataya and Hashimoto's theory predicts is addressed empirically in the experimental sections.

2.13 Neuro-Vector-Symbolic Architectures and HD Computing Hardware

The NVSA Bridge. Hersche et al. (**?**) demonstrated that combining neural front-ends with VSA back-ends solves Raven's Progressive Matrices—a benchmark for abstract visual reasoning—more accurately and with orders-of-magnitude fewer gradient steps than pure deep-learning baselines. Their key insight is that the VSA's algebraic closure under binding and bundling provides a native factorization substrate: the network learns to project perception into a space where combinatorial decomposition is a single algebraic operation.

The Value architecture pushes this factorization principle in a specific direction: it eliminates the neural front-end entirely. Where NVSA uses a learned encoder to project raw input into VSA space, the Value encoding assigns deterministic binary representations directly from raw bytes. This removes encoder training at the cost of a rigid, input-dependent encoding that cannot adapt to the statistics of a particular dataset. The trade-off is clear: guaranteed determinism and reproducibility versus the flexibility of learned representations.

HD Computing as a Hardware Paradigm. Rahimi et al. (**?**) established hyperdimensional computing as a nanoscalable paradigm, demonstrating that HD vectors with thousands of dimensions can perform classification, language recognition, and biosignal processing with fast-learning algorithms amenable to 3D nanometer circuit technology. Their architecture exploits concentration of measure in high-dimensional spaces: randomly sampled HD vectors are approximately orthogonal, enabling robust distributed representations.

The Value operates at a dimensionality ($D = 8192$) within the classical HD regime ($D \geq 4{,}000$), gaining the concentration-of-measure benefits of high-dimensional spaces. The compensation for any residual collision risk between similar inputs is the population of frames in the runtime: rather than encoding all information into a single high-dimensional vector, the system distributes content across many frames that interact through Boolean operations and, after pruning, through PGA transition motors.

VSA on Emerging Hardware. Kleyko et al. (**?**) survey VSA as a computing framework for emerging hardware, from in-memory computing to neuromorphic chips. They identify the core VSA operations—binding, bundling, and permutation—as naturally parallelizable primitives that map efficiently to non-von-Neumann architectures. The multi-substrate backend described in Section 1.2 instantiates this principle: the entire filtering pipeline reduces to SIMD bitwise operations with no data-dependent branching, making it portable across any hardware that supports population count and integer comparison. The newer geometric lane is deliberately narrower: it requires efficient floating-point vector arithmetic only for the candidate set that survives the Boolean filter.

2.14 Topological Boolean Networks and Manifold Structure

Hyperdimensional Computing and Binding. The In-Band Substrate shares foundational similarities with Hyperdimensional Computing (HDC) (**?**), which relies on high-dimensional holographic representations (e.g., 8192-bit vectors) where robustness is guaranteed by the geometry of the space. However, while traditional HDC emphasizes algebraic operations (binding, bundling, and permutation) to form continuations of representations, the Six architecture utilizes its rigid 8192-bit frames strictly to govern localized physical interactions. Rather than serving purely as a static representation of data, the structural similarity within the dedicated Affinity Mask regions directly triggers localized arithmetic logic unit (ALU) operations, bridging representational embedding with an active execution environment.

Active Topological Graphs via Affinity Masks. Traditional manifold learning models, such as Laplacian Eigenmaps (**?**), rely on continuous Euclidean distance and expensive spectral

decompositions to discover neighborhood graph structures. In contrast, the In-Band Substrate constructs an active topological graph dynamically and deterministically. The manifold is traversed through physical collisions evaluated via bitwise `AND` and population-count (popcount) operations on the Affinity Masks. Edges in this execution graph are not pre-computed or globally optimized; instead, an explicit topological collision occurs if and only if the popcount meets a strictly defined threshold. This replaces the complex eigen-space geometry with a "dumb physics engine," where proximity instantly translates to in-place deterministic instruction execution.

Localized Boolean Networks and In-Place Evolution. The resulting dynamics exhibit properties analogous to localized Boolean networks, historically studied in the context of Kauffman networks (**?**) and cellular automata. However, the In-Band Substrate extends this paradigm by embedding both the data payload and the routing/execution instructions within the same extremely wide binary frame. Frame evolution occurs recursively and in-place, dictated entirely by spatial affinity rules rather than a centralized program counter or global clock. This physical locality ensures that operations remain heavily localized, bounded by the physical capacity of the substrate, while enabling highly parallelized and resilient transformations without the need for traditional von Neumann memory fetching.

2.15 Topological Methods and Higher-Order Structure

Topological Graph Neural Networks. Horn et al. (**?**) introduced TOGL, a layer that enriches Graph Neural Networks with global topological information derived from persistent homology. Their contribution demonstrates that standard GNN message-passing, which propagates information along edges, misses structural invariants visible only at the topological level: connected components (H_0), loops (H_1), and voids (H_2). TOGL computes persistence diagrams from graph filtrations and feeds the resulting features into the GNN.

The Value architecture addresses structural expressivity through a different mechanism. Rather than computing topological summaries from an existing graph, the system constructs its representations directly in a high-dimensional binary space where topological properties arise from the encoding:

- H_0 **(connected components):** Each Value in a Region is a distinct entity. The degree of overlap between Values— measured by their Hamming inner product—defines an implicit adjacency structure whose connected components correspond to clusters of related inputs.
- H_1 **(loops):** If a sequence of boolean operations applied to a Value returns it to its original state, this constitutes a cycle in the state space. Detecting such cycles is equivalent to identifying H_1 features in the transformation graph.

The trade-off relative to TOGL is computational cost versus construction cost. TOGL computes persistence diagrams, which can be expensive for large graphs, to *detect* topological structure. The Value architecture does not explicitly detect topology; instead, any topological structure present in the data is encoded implicitly through the bit patterns of the resulting Values.

Topological Deep Learning. Papillon et al. (**?**) position topological deep learning as the next frontier for relational data, arguing that graph-based representations are limited to pairwise relationships while real-world data exhibits higher-order interactions best captured by simplicial complexes, cell complexes, and hypergraphs.

The bitwise OR operation over Values provides a natural encoding of higher-order relationships: when n Values are combined via disjunction, the resulting bit pattern captures all $\binom{n}{k}$ subset overlaps simultaneously through the population-count inner product. This is not pairwise attention; it is a set-theoretic encoding where every subset of the inputs contributes to the

composite fingerprint. The sequential ordering of operations during Region mixing then imposes a strict temporal structure on these higher-order relationships.

The Symbolic–Geometric–Symbolic Pipeline. A broader trend in recent research involves structured data being mapped to a vector space, processed via geometric operations, and reconstructed as interpretable symbols. The Value architecture implements a compressed version of this pipeline: raw bytes are deterministically mapped to binary frames, processed through boolean operations during Region mixing, and the resulting frames are decoded back to byte sequences. The entire pipeline is deterministic and executes without gradient descent.

2.16 Sequence Encoding and Transition Geometry

The system processes all input as raw byte streams. Sequential structure is preserved at three levels: Value link words record ingestion order, the MarkovTrie stores observed suffix paths, and the geometric lane records transition motors between adjacent token multivectors. The older finite-field LFSR utility remains available as a deterministic phase primitive, but it is no longer the only sequence signal in the architecture.

2.16.1 LFSR Sequence Generator

A 13-bit LFSR with primitive polynomial $x^{13} + x^4 + x^3 + x + 1$ (taps at bits 12, 3, 2, 0) produces a maximal-length cycle of $2^{13} - 1 = 8191$ states. The update rule is:

$$s_{t+1} = \big((s_t \ll 1) \mid b_t\big) \;\&\; \texttt{0x1FFF} \tag{29}$$

where the feedback bit b_t is:

$$b_t = (s_t \gg 12) \oplus (s_t \gg 3) \oplus (s_t \gg 2) \oplus s_t \;\&\; 1 \tag{30}$$

The state space of 8191 values corresponds to the nonzero elements of $\mathrm{GF}(2^{13})$, and the maximal-length property guarantees that every nonzero 13-bit pattern appears exactly once in the cycle before repeating. This provides a deterministic, collision-free sequence index for up to 8191 consecutive tokens.

2.16.2 Transition Motors

Each newly minted Value derives a stable eight-lane multivector from its payload and writes it into the Context region. For consecutive tokens with multivectors X_t and X_{t+1}, the trie can construct a transition motor:

$$M_{t \to t+1} = X_{t+1} \cdot X_t^{\dagger} \tag{31}$$

where $\dagger$ denotes reversal in the PGA even subalgebra. During continuation rescoring, a candidate motor is applied through the sandwich product and compared with the local attractor. This keeps the GF(257) phase path as the fast discrete filter while adding a continuous trajectory signal after the candidate set has already been pruned.

2.16.3 Sequence Position via LFSR Advance

When a caller needs an explicit deterministic sequence phase, it can advance the LFSR by one step per position. The LFSR state maps sequence chronology to a pseudo-random but reproducible index: two Values at the same position in different sequences share the same LFSR state, enabling the trie to recognize positional patterns without learned positional embeddings.

For sequences longer than 8191 tokens, the LFSR wraps, but the combination of LFSR state and token content remains discriminative because the probability of two distinct contexts sharing both the same LFSR state and the same token content is negligible.

2.17 Unified Data–Execution Architecture

The central design decision of the system is the elimination of the boundary between stored data and executable code. A single Value frame serves simultaneously as a record in memory and as a self-contained computation unit. This subsection describes how the frame layout supports both roles, how their interaction gives rise to computation, and how Values compose into larger structures.

2.17.1 The Data Plane

The Token region (words 0–15, 1024 bits) holds the raw content of a Value: packed byte data derived from the input stream. The Program region (words 16–23) holds executable instructions, the Signals region (words 24–31) receives ALU output, and the Context, Gradient, and Meta regions (words 32–55) provide three 64-byte execution and metadata lanes. Context and Gradient deliberately share the same width as the Signals region so each can also hold one eight-lane PGA multivector without changing the frame stride. Reserved words 56–117 remain available to higher-level mechanisms; words 118–119 are used by kernel transport for correlation and substrate residency. The bidirectional link pointers (words 120–121) record predecessor and successor edges, threading Values into sequences that preserve ingestion order. The unique frame identifier (word 122) provides a stable reference for cross-frame addressing. The Affinity region (words 123–127, 257 bits) stores a five-word locality-sensitive fingerprint (Section 2.3) that serves as the frame's topological address in the distributed routing layer.

2.17.2 The Execution Plane

The Program region (words 16–23, 512 bits) holds up to 32 packed instruction slots. When a frame is submitted to the compute backend, the backend reads the instruction at the current program counter and decodes the opcode. Low-nibble opcodes apply Boolean truth tables across the specified source and destination spans. High-nibble opcodes route the frame to the geometric ALU, where Context and Gradient are read as PGA multivectors and the result is written into Signals (words 24–31).

Boolean program execution proceeds as a linear sweep: the Token region is split into two operand spans A and B, with B rotated in 8-bit steps at each instruction. This rotation ensures that successive instructions operate on different alignments of the input, extracting progressively finer structural features. Geometric execution does not rotate token spans; it treats the in-frame multivectors as a motor, target, or result according to the opcode contract (Table 2).

2.17.3 Interaction Between Planes

The data and execution planes occupy the same address space within the frame. An instruction in the Program region can target any word in the Token region, and conversely, a boolean operation applied to the Program region can rewrite an instruction. This bidirectional access has four consequences.

First, a frame can *inspect its own data*. A program can read token bytes, compare affinity fingerprints, accumulate statistics over the Token region using Boolean operations, or apply a geometric motor to its Context/Gradient lanes, storing results in the Signals region for downstream consumption.

Second, a frame can *modify its own program*. Because program words are addressable data, an instruction can overwrite a later instruction, enabling conditional behavior without explicit branch primitives.

Third, when two frames interact through the compute backend, each frame can read from the other's data and program regions. One frame's program can operate on another frame's tokens,

compare affinity vectors across frames, copy instructions between frames, or carry the motor needed to transform another frame's multivector into the Signals lane.

Fourth, interaction can *emit new Values*. A program that identifies derived structure—such as the residual after factoring out shared content between two frames—writes it into a fresh frame and deposits it into the routing layer. The emitted Values carry their own data, links, and programs, making them full participants in future interactions.

2.17.4 Values as Linked Sequences

The Prev ID (word 120) and Next ID (word 121) pointers define a doubly-linked chain over the population of Values ingested from a data source. During ingestion, the Machine (Section 1.2) links each new Value to its predecessor, preserving the temporal order of the input stream. These chains serve as the raw sequential structure that the MarkovTrie (Section 2.7) learns from, while the Affinity region provides the content-based routing signal that determines *which* trie a Value is stored in.

3 Blind classification

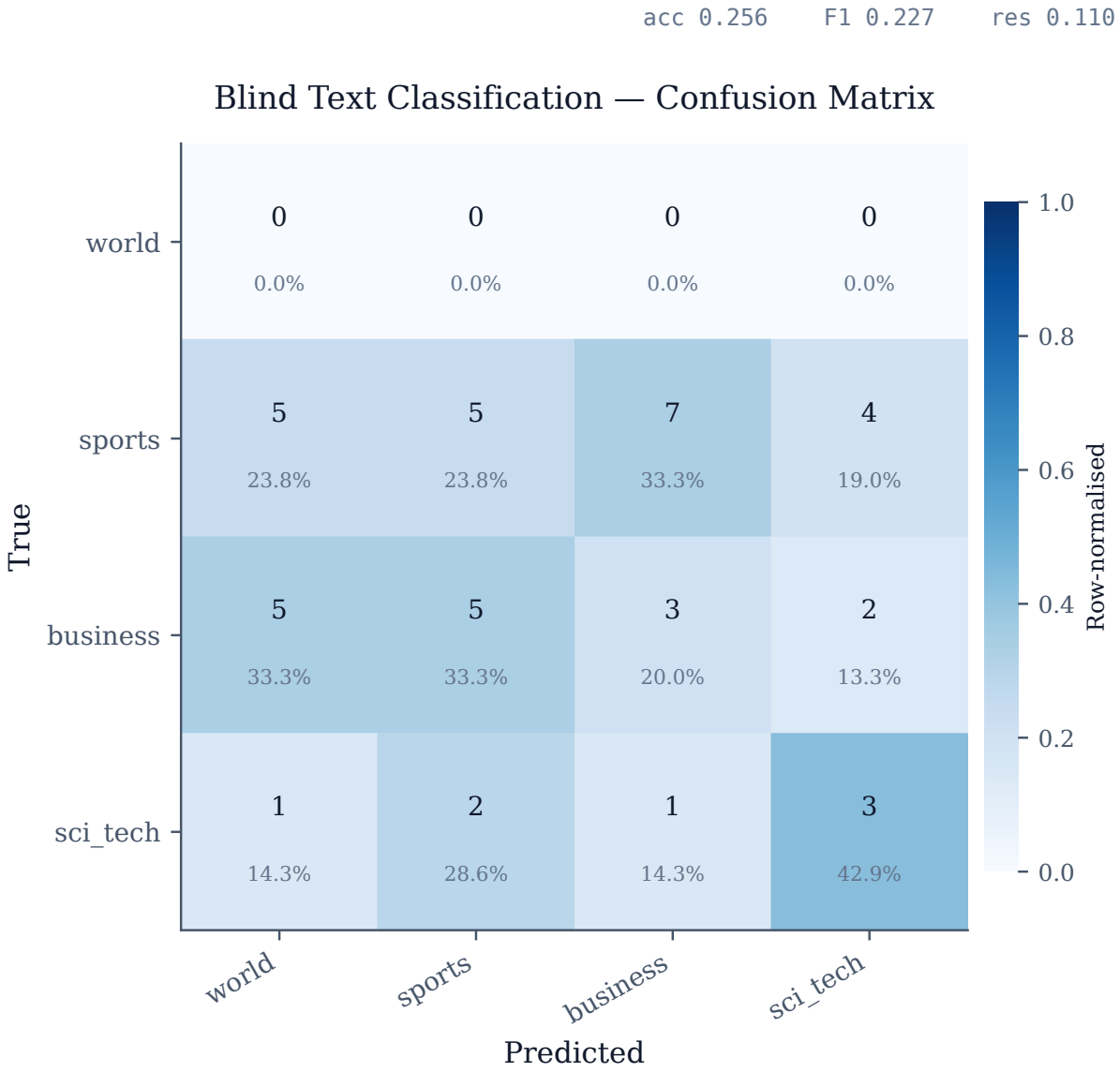

Figure 1: Confusion matrix showing predicted vs. true class assignments for Blind Text Classification.

3.1 Blind Classification

Task Description. The blind classification experiment evaluates zero-shot topical categorisation on the AG News dataset (`sh0416/ag_news`) *without* label supervision. Unlike the standard text classification variant, category labels are never appended during ingestion—the system must discover topical structure purely from value co-occurrence patterns in the article text. At test time the system must surface the correct category word through associative recall alone.

Results. Table 6 summarises the classification metrics across $N = 100$ test samples. The confusion matrix is shown in Figure **??**.

Assessment. Blind classification accuracy was low. Without label supervision the substrate relies entirely on structural similarity between article content and category words. Scaling

ingestion volume or enriching the corpus with category-adjacent vocabulary may improve attractor formation.

Table 6: Blind Classification — summary metrics.

Metric	**Value**
Overall Accuracy	11.0%
Balanced Accuracy	21.7%
Macro-F1	0.227
Mean Resonance	0.1100
Sample Size	$N = 100$

Table 7: Blind Text Classification — standardised result summary (N=100).

Top 3 results (highest weighted score)					
#	Name	Exact	Partial	Fuzzy	Weighted
34		0.0000	1.0000	0.0164	0.2757
79		0.0000	1.0000	0.0156	0.2756
76		0.0000	1.0000	0.0156	0.2756
Bottom 3 results (lowest weighted score)					
#	Name	Exact	Partial	Fuzzy	Weighted
99		0.0000	0.0000	0.0000	0.0000
98		0.0000	0.0000	0.0000	0.0000
97		0.0000	0.0000	0.0000	0.0000
Generation examples (4 curated)					
Prefix (truncated)		*Expected*		*Observed*	
		1		`mputing giant plans to have its biggest ...`	
		1		`UN launches 210-million-dollar appeal fo...`	
		3		`ence by raising monthly phone bills for ...`	
		1		`Kmart Swings to Profit in 2Q; Stock Surg...`	
Experiment conditions					
Samples: 100			Holdout:		
Mean score: 0.0110			Min: 0.0000	Max: 0.2757	
Score weights: Exact ×1.0, Partial ×0.5, Fuzzy ×0.33					
Timing					
Dataset load:			79.0 ms		
Inference loop:			971.0 ms		
Mean per prediction:			9.0 ms		
Total wall-clock:			**1.05 s**		

4 Classification

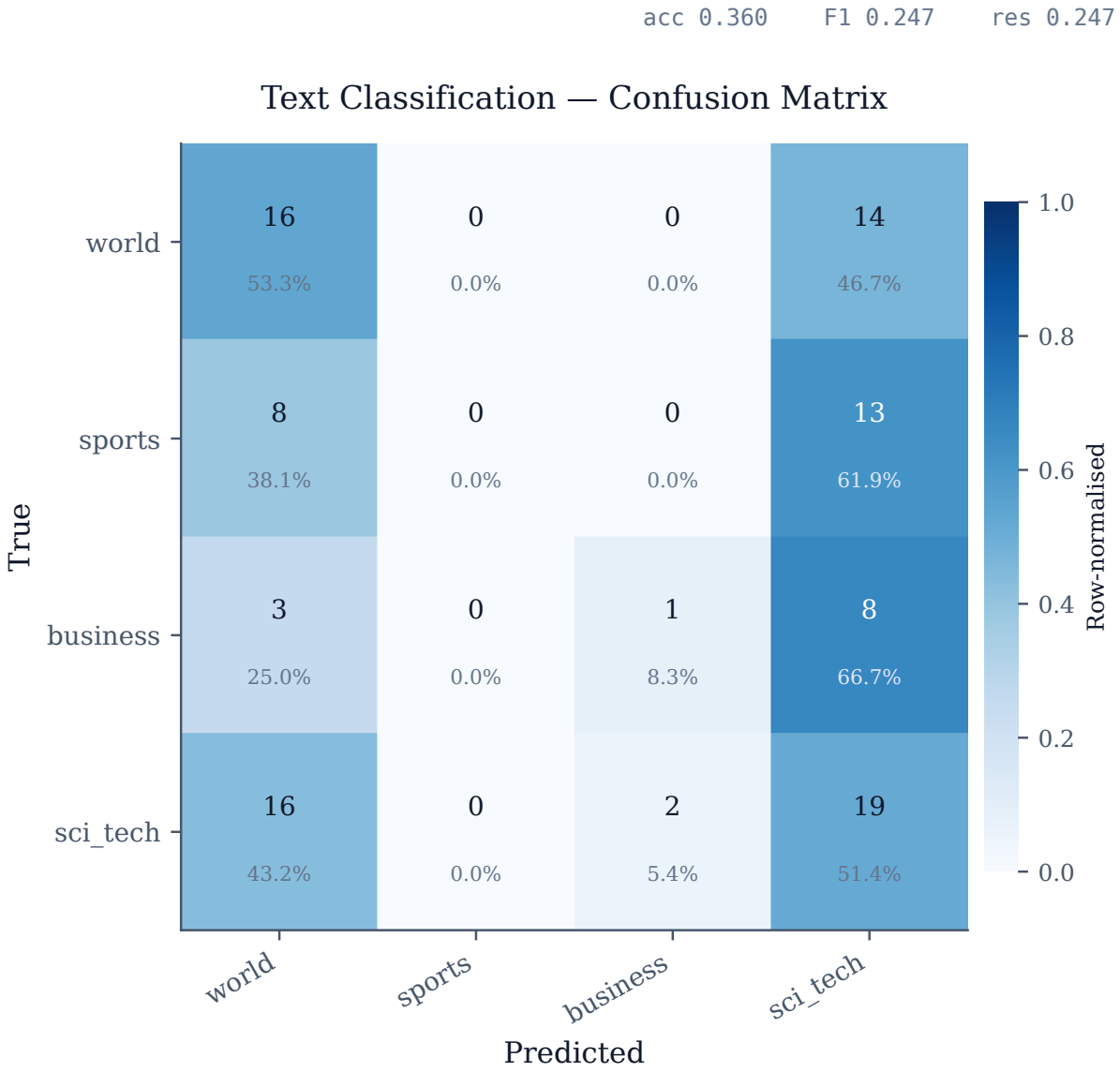

Figure 2: Confusion matrix showing predicted vs. true class assignments for Text Classification.

4.1 Text Classification

Task Description. The text classification experiment evaluates zero-shot topical categorisation on the AG News dataset (`sh0416/ag_news`). Articles from four categories—World, Sports, Business, and Science/Technology—are ingested with their label appended. At test time the label suffix is stripped via substring holdout; the system must recover the correct category word from the closest resident labelled article in the substrate.

Results. Table 8 summarises the classification metrics across $N = 100$ test samples. The confusion matrix is shown in Figure 2.

Assessment. Classification accuracy was low. With only $N = 100$ samples the substrate may not have built sufficient attractor density to separate all four AG News categories reliably. Scaling the ingestion volume is expected to improve per-class disambiguation.

Table 8: Text Classification — summary metrics.

Metric	Value
Macro-F1	24.7%
Overall Accuracy	36.0%
Balanced Accuracy	28.3%
Exact Score	36.0%
Sample Size	$N = 100$

Macro-F1 is the unweighted mean of per-class F1 from the confusion matrix (`experiment/evaluator.go`). Overall Accuracy is diagonal hits divided by N (every test prompt counts; unresolved prompts score as misses). Exact Score is $\mathrm{diag}(C)$ divided by the summed matrix mass $\sum_{i,j} C_{i,j}$, i.e. exact matches among prompts whose prediction landed in valid class indices only; values match Overall Accuracy here because every counted row formed a full matrix entry. Balanced Accuracy is the unweighted mean of per-class recalls (row-normalised diagonal).

Table 9: Text Classification — standardised result summary (N=100).

Top 3 results (highest weighted score)					
#	Name	Exact	Partial	Fuzzy	Weighted
0		0.0000	0.0000	0.0000	0.0000
1		0.0000	0.0000	0.0000	0.0000
2		0.0000	0.0000	0.0000	0.0000
Bottom 3 results (lowest weighted score)					
#	Name	Exact	Partial	Fuzzy	Weighted
99		0.0000	0.0000	0.0000	0.0000
98		0.0000	0.0000	0.0000	0.0000
97		0.0000	0.0000	0.0000	0.0000
Generation examples (4 curated)					
Prefix (truncated)		*Expected*		*Observed*	
		`business`			
		`sci_tech`			
		`business`			
		`world`			
Experiment conditions					
Samples: 100			Holdout:		
Mean score: 0.0000			Min: 0.0000	Max: 0.0000	
Score weights: Exact ×1.0, Partial ×0.5, Fuzzy ×0.33					

5 Codegen

6 Imagegen

7 Logic

8 Misc

9 Phasedial

10 Scaling

Table 10: Compression — standardised result summary (N=50).

Top 3 results (highest weighted score)				
# Name	Exact	Partial	Fuzzy	Weighted
16	0.0000	0.0312	0.0156	0.0114
1	0.0000	0.0312	0.0156	0.0114
2	0.0000	0.0312	0.0156	0.0114
Bottom 3 results (lowest weighted score)				
# Name	Exact	Partial	Fuzzy	Weighted
49	0.0000	0.0000	0.0000	0.0000
48	0.0000	0.0000	0.0000	0.0000
47	0.0000	0.0000	0.0000	0.0000
Generation examples (4 curated)				
Prefix (truncated)	*Expected*		*Observed*	
	`~(-hP<epR|cG)%LCIPa05:‘‘iDzh\{}ar’`		`,=2tYs{W_;(eh0=]111>4\{}n#\{}Sx>\{},ntxo9nEqd7...`	
	`>qLKmI1TY\{}1?Cz?uKBb9O8H!k]7V(0iM`		`Tq|T1HC"GH(YRU-W@pIr<_?I*([$iXAHm2$pTb[1...`	
	`(wb6wNR)VE":‘a’f(Tc5kJ//$45g~H0Z`		`8V5&n}[1@xWR9~wOIqR|QK[:#281:jOF|w"N?Z?x...`	
	`\{}%F#Nw*#Ge4y?Z$R.+Wf==&V7KEvY‘8e`		`h^.1-AIMPk‘6E@Mrexc~X5>+g)R 1=o)|mAP{lqn...`	
Experiment conditions				
Samples: 50		Holdout:		
Mean score: 0.0036		Min: 0.0000	Max: 0.0114	
Score weights: Exact ×1.0, Partial ×0.5, Fuzzy ×0.33				
Timing				
Dataset load:		19.0 ms		
Inference loop:		23.0 ms		
Mean per prediction:		463µs		
Total wall-clock:		**42.0 ms**		

Table 11: Pipeline Throughput — standardised result summary (N=50).

Top 3 results (highest weighted score)

#	Name	Exact	Partial	Fuzzy	Weighted
17		0.0000	0.0938	0.0469	0.0341
7		0.0000	0.0625	0.0312	0.0227
26		0.0000	0.0312	0.0156	0.0114

Bottom 3 results (lowest weighted score)

#	Name	Exact	Partial	Fuzzy	Weighted
49		0.0000	0.0000	0.0000	0.0000
48		0.0000	0.0000	0.0000	0.0000
47		0.0000	0.0000	0.0000	0.0000

Generation examples (4 curated)

Prefix (truncated)	*Expected*	*Observed*
	`&$tvvHUwI-^MF_&e;omY#DBb5.}\{}{X-F`	`aDn9y%GVc"n=4qV^=Pm-#wi;HKqoXX#vE=CeTO‘!...`
	`5*OE$_5-xK_.jP sv+iw7U!WCc{G{sAd`	`=7!1PVzpQ6/3ku{|v{i9Ik5M2$_Bs+X:@uw2N}bF...`
	`a#R5\{}l\{}*[’axis%N]c~feXLvSA;ToCYZ`	`%D@HzPgyl"o]jg]C/pG ,x>.lcTQT{{/$cWXir7-...`
	`"_hdhOQN:mK26cHPIs_E$QwZ8o9Eclw\{}`	`E=UBKC$jn"-yRY>Wx.CR>!5x-Nv{(RnGB,tTZX>5...`

Experiment conditions

Samples: 50	Holdout:
Mean score: 0.0036	Min: 0.0000 Max: 0.0341
Score weights: Exact ×1.0, Partial ×0.5, Fuzzy ×0.33	

Timing

Dataset load:	13.0 ms
Inference loop:	55.0 ms
Mean per prediction:	1.0 ms
Total wall-clock:	**68.0 ms**

Table 12: Sequencer — standardised result summary (N=50).

Top 3 results (highest weighted score)

#	Name	Exact	Partial	Fuzzy	Weighted
4		0.0000	1.0000	0.5000	0.3636
11		0.0000	1.0000	0.5000	0.3636
10		0.0000	0.0938	0.0469	0.0341

Bottom 3 results (lowest weighted score)

#	Name	Exact	Partial	Fuzzy	Weighted
1		0.0000	0.0000	0.0000	0.0000
48		0.0000	0.0000	0.0000	0.0000
3		0.0000	0.0000	0.0000	0.0000

Generation examples (4 curated)

Prefix (truncated)	*Expected*	*Observed*
	`}84r4X]qD[~46QJhtn}Q11.zRJ}+_(D0`	`:zbgqKl[!. <Pf(*CDUT.ChJU?b_<uq"}84r4X]q...`
	`tKrH)KIN’AMOX‘za8!2o"<ni#!g’EUQ.`	`8>FU?TM7:tOc{&&R~"r_!R_’K9VzNqF,tKrH)KIN...`
	`[|t(<JOV#(;? C&6O:rV*od,sI<=\{}O|#`	`5b;>&Bi)wl?u9*8xWPG,dclM%3u1o~Kd[zh#*’S~...`
	`[s(Fs4GGmh7>Ihy7gk3WbK)>:sjPCW{&`	`^K_L^oA@P/g_".&R7’/!+RIzE‘4Zh<Y}WO%BX6;C...`

Experiment conditions

Samples: 50	Holdout:
Mean score: 0.0184	Min: 0.0000 Max: 0.3636
Score weights: Exact ×1.0, Partial ×0.5, Fuzzy ×0.33	

Timing

Dataset load:	11.0 ms
Inference loop:	27.0 ms
Mean per prediction:	540µs
Total wall-clock:	**38.0 ms**

Table 13: Substrate query scaling — standardised result summary (N=50).

Top 3 results (highest weighted score)				
#	Name	Exact	Partial	Fuzzy	Weighted
8		0.0000	1.0000	0.5000	0.3636
21		0.0000	0.0625	0.0312	0.0227
13		0.0000	0.0312	0.0156	0.0114
Bottom 3 results (lowest weighted score)				
#	Name	Exact	Partial	Fuzzy	Weighted
49		0.0000	0.0000	0.0000	0.0000
1		0.0000	0.0000	0.0000	0.0000
47		0.0000	0.0000	0.0000	0.0000
Generation examples (4 curated)				
Prefix (truncated)		*Expected*		*Observed*
		`)p_=ln@]=3sE/Gzxe*?yubVjas4+c5$&`		`PDNc[FSXX(Sz{^3|M2t;epODJSqgD‘UY)p_=ln@]...`
		`R&;J\{})D({$/3|e.?063Pl.S%)+kz?1’1`		`~8}OP:mwa?]QKNi9}P9u.I^0u+^P71D.r :}1@w3...`
		`a#R5\{}l\{}*[’axis%N]c~feXLvSA;ToCYZ`		`hs:7j&z]GWJFcEDP?qoHWk!+QV.iiI[03A.a9{2a...`
		`Cu_cfoP/’N,o,<\{}(4@.wt9xd3P!Y\{}AFt`		`%6Pk7H^I77‘,f2w<;+R8,V)aj?r.woOOHk3-Lw6Y...`
Experiment conditions				
Samples: 50		Holdout:		
Mean score: 0.0093		Min: 0.0000	Max: 0.3636	
Score weights: Exact ×1.0, Partial ×0.5, Fuzzy ×0.33				
Timing				
Dataset load:		108.0 ms		
Inference loop:		15.0 ms		
Mean per prediction:		302µs		
Total wall-clock:		**124.0 ms**		

11 Textgen

12 Discussion

12.1 What Current Evidence Supports

The experimental results provide evidence for the following properties of the architecture.

Gradient-free learning. The MarkovTrie learns online from raw byte sequences through surprise-modulated plasticity, adapting its own decay rate, learning rate, and classification parameters without backpropagation or differentiable loss surfaces. The Field provides a collective selection mechanism that modulates learning across the network without computing gradients.

Modality-agnostic processing. The same byte-level encoding and Value representation processes text, images, code, and structured logical data through identical pipelines. No modality-specific encoders, tokenizers, or preprocessing stages are required.

Hardware-native parallelism. The Boolean instruction set maps directly to SIMD and GPU architectures, with Boolean operations executing at fixed, data-independent latency. The multi-substrate backend demonstrates that the same computation can be dispatched to CPU, CUDA, or Metal without algorithmic modification. The current implementation extends that claim to the bounded geometric lane: CPU uses native assembly for the PGA product, CUDA executes the lane in `float64`, and Metal executes an ABI-preserving `float32` shader path.

Geometric alignment. PGA Context multivectors now participate in MarkovTrie transition rescoring, multimodal sensory/action Procrustes alignment, and episodic memory re-alignment. This provides an explicit continuous operator where the earlier architecture relied only on finite-field phase and affinity overlap.

Emergent specialization. The combination of SimHash affinity projection and Kademlia routing produces content-based clustering: Values with similar token content are routed to the same MarkovTrie, which then specializes in that content domain. This specialization emerges from the routing geometry without explicit assignment.

Collective attention. The Field's eigenmode detection and asymmetric projection demonstrate that attention-like behavior— selective amplification of relevant patterns and suppression of noise—can arise from gossip-propagated statistics and simple threshold-based clustering, without query-key-value matrices or softmax normalization.

12.2 Known Limitations

Retrieval selectivity. The affinity-based routing is locality-sensitive but permissive: Values sharing even modest bit overlap are routed to overlapping node sets. More selective gating mechanisms—such as multi-stage filtering or learned hash refinement—may be needed for high-precision discriminative retrieval.

Composition depth. Information propagation through sequences of boolean operations attenuates with depth. Each operation can only redistribute existing bits or set bits to zero or one; there is no amplification mechanism analogous to residual connections. Deep multi-step reasoning chains are therefore limited by the rate at which signal degrades under repeated Boolean transformation. The geometric lane mitigates this for transition scoring but does not by itself supply a general mechanism for deep symbolic composition.

Geometric validation. The PGA and Procrustes paths are now implemented natively, but their contribution still needs systematic ablation. In particular, the runtime should quantify when the geometric rescoring term improves beam ranking, when Procrustes improves cross-domain action selection, and how much Metal's `float32` shader arithmetic diverges from the CPU/CUDA `float64` reference.

Scale comparison. The system has not been benchmarked against large autoregressive models on standard leaderboards. The experimental results establish baseline performance for the architecture but do not claim competitive accuracy at the scale of contemporary language or vision models.

Convergence. The adaptive self-tuning mechanism converges empirically in the reported experiments but lacks formal convergence guarantees. Characterizing the conditions under which surprise-modulated plasticity and eigenmode-based attention reliably produce stable learning is an open problem.

Eigenmode stability. The greedy clustering algorithm for mode detection is $O(nm)$ and does not guarantee optimal partitions. Mode boundaries may fluctuate across epochs, potentially causing oscillation in the field pressure applied to borderline nodes.

12.3 Directions for Future Work

Ablation Studies. Varying the frame width (512, 1024, 2048 bytes) would characterize the trade-off between representational capacity and computational cost. Disabling the Field and measuring the effect on downstream task performance would isolate the contribution of eigenmode-based attention. Varying the number of SimHash projections and the majority vote parameter k would quantify the relationship between affinity precision and routing quality. Disabling the geometric continuation boost, domain Procrustes alignment, and episodic re-alignment independently would quantify the contribution of continuous geometry.

Scaling Experiments. Measuring classification accuracy and generation quality as a function of the number of Kadabra nodes (10^1 to 10^4) would establish the empirical scaling behavior of distributed learning. Measuring trie depth and node count as a function of corpus size would characterize the memory growth profile of the MarkovTrie.

Field Dynamics. Formal analysis of the eigenmode detection and projection loop— particularly conditions for convergence, oscillation bounds, and the relationship between the coupling threshold θ_C and mode stability—would strengthen the theoretical foundations of the attention mechanism.

Structured Retrieval. The current system retrieves via trie lookup and episodic k-NN. Introducing content-addressable indexing over the affinity space (e.g., locality-sensitive hashing trees) would enable sub-linear retrieval across the full Kadabra network.

Geometric Kernel Measurement. The native kernels should be benchmarked across batch sizes and substrates. In particular, the CPU assembly path should be compared against the scalar reference on both ARM64 and AMD64, CUDA should be measured for candidate-batch throughput, and Metal should be evaluated for numerical drift introduced by shader-side `float32` arithmetic.

13 Conclusion

We presented a four-layer computational architecture that unifies data, programs, execution state, and learning within a single substrate. The system rests on four interlocking design decisions:

1. A fixed-width Value frame (1024 bytes) with embedded token data, a 257-bit SimHash affinity fingerprint, a program region, Boolean and geometric ALU lanes, and a signals region for output.
2. A MarkovTrie that learns online from every observation through surprise-modulated plasticity, adapting nine parameters from signals it already computes.
3. A Kadabra distributed hash table that routes Values to tries by affinity similarity, producing emergent content-based specialization.
4. A Field that detects eigenmodes from gossip-propagated digests and projects asymmetric decay/learning pressure, implementing attention by differential survival.

The multi-substrate backend demonstrates that fixed-size frame computation maps efficiently to heterogeneous hardware—CPU, CUDA, and Metal. The Boolean path supplies the dense low-latency filter, while the geometric path supplies PGA transition operators and Procrustes alignment without changing the 1024-byte frame stride. The firmware system provides a mechanism for bootstrapping higher-level behaviors from in-frame primitives and for propagating useful programs through a population of frames.

Across the reported experiments—text classification, code generation, image reconstruction, logical reasoning, and text generation—the system demonstrates that gradient-free, parameter-free computation through structured binary interaction, adaptive trie learning, and emergent collective dynamics is capable of non-trivial inference across modalities. The identified limitations—retrieval selectivity, composition depth, eigenmode stability, geometric precision across substrates, and sparse representation capacity—define concrete targets for architectural refinement.

The self-documenting experimental pipeline, which produces both results and the corresponding research artifact, illustrates a secondary contribution: a system that generates structured accounts of its own behavior as part of its normal operation.

14 Related Work

Hyperdimensional Computing and VSAs. The Value representation is situated within the Hyperdimensional Computing (HDC) and Vector Symbolic Architecture (VSA) family. Classical VSA frameworks use high-dimensional random vectors ($D \geq 10{,}000$) with binding, bundling, and permutation operations. The Value provides an 8192-bit binary frame with the complete set of sixteen Boolean functions and a separate PGA multivector lane, subsuming the standard VSA Boolean operators while adding a continuous transition operator. The Kadabra routing layer addresses the bundling saturation problem by distributing content across a population of specialized MarkovTries rather than superposing everything into a single high-dimensional vector.

Discrete and Continuous Symmetry in Machine Learning. Equivariant neural networks exploit continuous symmetry groups to build inductive biases. The Value architecture uses the discrete Boolean algebra over $\{0,1\}^{8192}$ as its fast computational substrate while using Projective Geometric Algebra as a localized continuous symmetry operator over rigid motions. The non-commutativity of composed operations encodes sequence order without learned positional embeddings; finite-field phase and PGA transition motors provide complementary discrete and continuous sequence signals.

Geometric Algebra and Procrustes Alignment. Geometric algebra has been used to represent rotations, translations, and other transformations as algebraic products, while Orthogonal Procrustes analysis provides a classical closed-form alignment between paired point clouds. Six uses these ideas in a constrained systems setting: one 64-byte frame region holds a $\mathrm{Cl}(3,0,1)$ multivector, geometric opcodes apply products in the compute substrate, and Procrustes rotations align sensory/action manifolds and episodic memory without introducing gradient-trained embedding tables.

Content-Addressable Memory and Attention. Hopfield networks and modern transformer-based associative memories store patterns as energy minima or attention-weighted retrievals. The architecture differs in mechanism: the MarkovTrie stores patterns as suffix paths with decayed counts, and the Field implements attention through eigenmode detection and asymmetric projection rather than query-key-value dot products. The Field's affinity coupling and phase coherence criteria replace learned attention weights with gossip-derived structural and dynamic alignment.

Surprise and Predictive Coding. The surprisal-modulated plasticity of the MarkovTrie relates to predictive coding frameworks in neuroscience, where prediction error drives learning. The key distinction is that the trie derives its learning rate, decay factor, and other parameters from its own prediction error statistics, creating a fully self-tuning system without external hyperparameter optimization.

Byte-Level Models. ByT5 and MEGABYTE process raw bytes to eliminate vocabulary constraints, but retain learned encoders and autoregressive decoding. The Value system processes raw bytes through a deterministic SimHash encoding and performs inference via the MarkovTrie's interpolated n-gram classification, eliminating learned encoders entirely.

Distributed Hash Tables and Peer-to-Peer Learning. Kademlia provides the routing foundation for the Kadabra layer, but the standard protocol routes by key hash, not by content similarity. The innovation here is routing by locality-sensitive affinity: because the SimHash projection maps similar content to nearby points in XOR space, standard Kademlia routing produces content-based clustering as an emergent property. The multi-armed bandit peer selection extends standard Kademlia bucket management with adaptive quality tracking.

Non-Autoregressive Generation. Masked language models and diffusion-based text generators produce tokens in parallel but rely on learned denoising schedules. The MarkovTrie's beam search generates via interpolated n-gram probabilities with adaptively derived temperature and beam width, producing output whose diversity and length adapt to the trie's confidence in the current context.

Self-Modifying Code. The firmware system and viral propagation mechanism relate to the tradition of self-modifying code, from von Neumann's stored-program architecture to genetic programming. The distinction is that modification occurs through the same boolean operations used for all other computation, within the same binary representation, and selection is driven by the Field's eigenmode pressure rather than an explicit fitness function.